Invention Disclosure Format (IDF)-B

Document No. 02-IPR-R003

Issue No/Date 2/01.02.2024

Amd. No/Date 0/00.00.0000

1. Title of the invention

NOVA: An Evidence-Grounded AI Security Intelligence Platform with Contradiction-Aware Natural Language Inference, Multi-Dimensional Trust Calibration, Decoupled Hard Safety Policy Gating, and Context-Aware Security Risk Verification

2. Field / Area of the invention

The disclosed invention belongs to the intersection of artificial intelligence, natural-language processing, cybersecurity intelligence, and knowledge-graph-based reasoning.

More specifically, the field concerns:

(a) Retrieval-augmented generation (RAG) systems that acquire, contextualize, and reason over heterogeneous evidence sources before permitting AI-generated output.

(b) Natural-language inference (NLI) applied to technical and security evidence, where semantic comparison must distinguish genuine evidence conflicts from apparent contradictions arising from differences in software version, security property, temporal context, or remediation state.

(c) Multi-dimensional trust and confidence calibration, in which multiple independent evidence-quality dimensions are combined and mapped to a calibrated probability rather than relying on a single similarity or model-confidence score.

(d) Explainable safety-gate mechanisms that impose discrete policy boundaries on AI-generated output, preventing high-confidence conclusions from being treated as automatically authoritative when safety-relevant evidence conditions are not satisfied.

(e) Security intelligence involving analysis of source-code and system structure using assets, observations, evidence, controls, trust boundaries, risk scenarios, and remediation states. These elements are represented in a security context graph so that a security conclusion is associated with the technical environment in which the observation occurs.

(f) Trust calibration and explainable response control. Multiple evidence-quality dimensions are converted into a calibrated trust assessment, after which an explicit safety policy determines whether a conclusion is generated, qualified, redirected to a fallback mechanism, or refused.

The disclosed field consequently covers the specific interaction between evidence acquisition, contextual NLI reasoning, multi-dimensional trust estimation, controlled AI generation, security-context graph analysis, and verified remediation.

3. Prior patents and publications from literature

A formal patent and publication search is required before this section is finalized. The following prior-art domains are relevant to the invention and should be investigated.

3.1 Prior-art categories

- Natural-language-inference systems for contradiction, consistency, entailment, and evidence comparison.
- Version-aware or context-aware contradiction detection in technical and security information.
- Retrieval-augmented generation (RAG), corrective/adaptive RAG, and evidence-grounded generation.
- Multi-factor confidence/trust calibration and safety/refusal gating for AI systems.
- Cybersecurity vulnerability aggregation, risk scoring, security posture management, and security intelligence.
- Cybersecurity knowledge graphs, graph-based reasoning, and GraphRAG.
- Automated remediation verification using source-code, static-analysis, or AST evidence.
- Knowledge-gap detection and feedback-driven knowledge-base or retrieval evolution.

3.2 Suggested search phrases

"NLI contradiction detection version disambiguation security evidence"; "multi-dimensional trust calibration LLM safety gate"; "security context graph risk scenario verification"; "AST remediation verification cybersecurity"; "evidence fusion RAG security intelligence"; "knowledge gap feedback adaptive retrieval".

3.3 Prior-art comparison table

Prior-Art Category	Representative References	Relation to NOVA	Gap / Distinction
Enterprise RAG pipelines	Google US12135740B1 (2024)	Discloses context validation in RAG retri	Does not disclose property/version NLI disambiguation, du
SAST / Code analysis	Microsoft US20230385451A1 (2023)	Discloses code analysis graph and vulne	Does not disclose NLI pairwise contradiction matrix, 8D tru
Trust calibration for AI	IBM US20240037256A1 (2024)	Discloses trust scoring and confidence c	Does not disclose AST security observations, property/vers
Security context graphs	Palo Alto Networks US11842186B2 (202	Discloses security context graph for thre	Does not disclose RAG evidence fusion, pairwise NLI cons
Vulnerability verification	Datadog WO2024015820A1 (2024)	Discloses vulnerability verification and re	Does not disclose dual-track RAG fusion, property/version
Standard NLI models	DeBERTa, RoBERTa, cross-encoder NL	Generic textual entailment/contradiction	Do not perform security-property-aware or software-versio
Corrective / Adaptive RAG	CRAG and Self-RAG literature	Discloses retrieval correction and self-ev	Do not fuse structured AST security observations with unst
AI safety / refusal	Constitutional AI, RLHF guardrails	Content-safety-focused refusal mechani	Do not couple refusal decision to pairwise NLI evidence co

3.4 Combination analysis

No specific patent or publication identified to date discloses substantially the same interaction among: (1) dual-track evidence acquisition with structured AST code observations and unstructured RAG knowledge, (2) property/version/temporal-aware NLI contradiction disambiguation, (3) eight-dimensional trust calibration with Platt scaling, (4) a decoupled hard safety policy gate that overrides statistical trust when active evidence contradictions are detected, (5) context-aware security intelligence with verified risk scenarios and three-tier remediation verification, and (6) closed-loop knowledge-gap evolution.

The existence of known techniques for NLI, RAG, confidence estimation, security graphs, static analysis, or refusal control does not by itself establish whether the complete disclosed processing flow has previously been described. Formal novelty and inventive-step conclusions require professional patent counsel review of the relevant patent and publication records.

The potentially important contribution is the specific interaction, ordering, architecture, and decision flow of the mechanisms, not generic claims to AI, RAG, NLI, graphs, or cybersecurity.

4. Summary and background of the invention

4.1 Technical problem

Modern cybersecurity decision systems operate over heterogeneous evidence including asset observations, security findings, technical artifacts, organizational knowledge, source-code and static-analysis information, and retrieved documents. Such evidence can be incomplete, stale, duplicated, version-dependent, or contradictory. A conventional retrieval-augmented or language-model-based response may retrieve statements that disagree without reliably

determining whether the disagreement is a genuine contradiction or an artifact of context.

4.2 Limitations of conventional systems

The following limitations are identified in existing approaches:

(a) Contradiction misclassification. Standard NLI classifiers label any opposing statements as contradictions. In technical cybersecurity documentation, "Package v1.0 is vulnerable to CVE-2024-1234" and "Package v1.2 has patched CVE-2024-1234" appear contradictory to a generic NLI model but actually represent a valid version upgrade path. Without property/version/temporal disambiguation, the system either (i) incorrectly refuses to answer, or (ii) merges conflicting information into an unreliable response.

(b) Single-dimensional trust. Conventional RAG systems use a single cosine similarity or model-confidence score to determine whether to generate. A high retrieval score does not necessarily indicate that evidence is fresh, mutually consistent, or drawn from reliable sources. An aged document with high lexical overlap may produce a high similarity score while the underlying information is outdated.

(c) Absence of evidence-decoupled safety gating. In conventional systems, if the model confidence or retrieval similarity exceeds a threshold, generation proceeds. There is no mechanism by which a discrete safety policy can block generation when evidence contradictions exist regardless of how high the statistical confidence appears.

(d) Isolated security findings. Standard SAST and vulnerability scanners produce isolated findings without contextual relationships among assets, controls, trust boundaries, and risk scenarios. Treating every finding independently prevents the system from reasoning about whether controls exist, whether risks are verified, or whether remediation has been validated.

(e) Unverified remediation. Recommending a fix does not demonstrate that a vulnerability or condition has been corrected. Without subsequent technical evidence verifying the actual code state after remediation, the system cannot distinguish an open vulnerability from a resolved one.

4.3 NOVA's approach

NOVA (Neural Orchestrated Vector Assistant) addresses these limitations through an integrated evidence-to-security-intelligence workflow. The system combines: contradiction-aware natural-language inference and consensus analysis with property and version disambiguation; multi-dimensional evidence-trust calibration with an explainable safety gate; a security context graph; control and risk analysis; risk-scenario generation and verification; remediation verification; and fusion of structured technical evidence with retrieved unstructured knowledge.

The technical contribution is centered on the interaction of these mechanisms. Evidence is interpreted in context before a conclusion is formed; contradictions are qualified rather than treated as simple textual disagreement; trust is calibrated using multiple evidence-quality dimensions; unsafe or insufficiently supported conclusions are gated; security relationships are represented in contextual form; risk scenarios and remediation states are verified; and the resulting intelligence is returned together with supporting evidence and an auditable decision record.

5. Objectives of the invention

(a) Generate security intelligence grounded in traceable evidence. The system retains a usable relationship between a conclusion and its supporting evidence, including both retrieved knowledge and structured technical observations, so that the basis of an intelligence result can be inspected.

(b) Detect and interpret contradictions while distinguishing genuine semantic conflict from property and version differences. The system compares evidence semantically while considering the security property, component, software version, and technical state to which each statement applies, reducing incorrect classification of contextual differences as contradictions.

(c) Calibrate confidence and trust using multiple independent evidence-quality dimensions. The system combines several evidence-quality signals rather than allowing one similarity or model-confidence value to determine whether an answer is trustworthy enough for downstream use.

(d) Prevent or qualify unsafe and low-confidence AI conclusions through an explainable safety gate. The system imposes an explicit policy boundary after evidence and trust analysis, allowing generation only when the required conditions are met and otherwise supporting warning, fallback, or abstention/refusal behavior.

(e) Construct a security context connecting assets, observations, evidence, controls, risks, and related entities. The system represents relationships among security-relevant objects so that observations can be interpreted in their architectural context and can inform subsequent control and risk analysis.

(f) Generate, verify, and contextualize security risk scenarios. The system distinguishes a generated scenario from a verified condition by applying contextual and verification logic before treating a scenario as supported security intelligence.

(g) Verify remediation using subsequent technical or structured evidence rather than relying only on generated recommendations. The system uses subsequent technical evidence to determine whether a security-relevant condition has actually changed, rather than treating the existence of a recommended fix as proof of remediation.

(h) Fuse structured security and static-analysis evidence with unstructured retrieved knowledge. The system provides a common evidence-fusion stage in which structured observations and unstructured knowledge can jointly influence downstream reasoning while retaining their respective provenance and context.

(i) Use observed knowledge gaps and feedback to support improvement of knowledge and retrieval behavior. The system converts suitable low-confidence or unanswered interactions into structured improvement signals that support curation, knowledge evolution, and deterministic handling of recurring queries.

6. Working principle of the invention (in brief)

6.1 Overall NOVA architecture

The NOVA platform implements a multi-layer processing architecture consisting of: (1) a frontend and API gateway tier; (2) a dual-track evidence ingestion and fusion subsystem; (3) a contextual NLI disambiguation and pairwise consensus engine; (4) an eight-dimensional trust calibrator with a decoupled hard safety override gate; (5) a multi-provider LLM gateway and streaming streamer; (6) a persistent storage tier; and (7) a closed-loop self-evolving knowledge subsystem.

6.2 End-to-end processing flow

The principal processing flow is:

Observations / Inputs

-> Asset & Observation Collection (AST code parsing of routes, inputs, secrets, DB queries)

-> Evidence Acquisition (dual-track: structured security intelligence + unstructured RAG knowledge)

-> Evidence Contextualization (security context graph: assets, controls, trust boundaries)

-> NLI / Consensus Analysis (pairwise semantic comparison across all evidence items)

-> Property & Version Disambiguation (7 security property types x 2 states, version comparison, temporal context)

-> Eight-Dimensional Trust Calibration (8D feature vector -> Platt logistic scaling -> P(Trust))

-> Explainable Safety Gate (hard contradiction override -> GENERATE / WARNING / FALLBACK / ABSTAIN)

-> Security Context Graph Enrichment (control and risk analysis using contextual relationships)

-> Risk Scenario Generation & Verification (candidate hypotheses verified against control evidence)

-> Remediation Verification (three-tier: textual presence, structural implementation, control effectiveness)

-> Security Posture / Trend Analysis (temporal delta dS = S_t - S_{t-1}, risk evolution classification)

-> Evidence-Grounded Explanation (provenance citations, safety metadata, auditable decision record)

The process begins when an authorized user query or security-analysis operation enters the application. The API layer provides a controlled boundary between the user-facing application and internal services, while authentication, role-based access controls, request context, and operational controls determine how the request is processed.

Evidence is then acquired through complementary paths. The security-intelligence path obtains technical observations from the source repository and associated structures. The knowledge path obtains enterprise information through document processing, indexing, vector retrieval, and related knowledge mechanisms. The two paths provide different but complementary evidence rather than duplicate copies of the same information.

In the knowledge path, documents are normalized and processed into structure-aware chunks. Heading and section information is retained as contextual metadata, and embeddings support semantic retrieval. Candidate knowledge items

are subsequently treated as evidence that participates in the reasoning process rather than being passed directly to generation without evaluation.

In the security-intelligence path, asset discovery and observation collection identify endpoints, APIs, models, services, and other security-relevant structures. These observations are represented in a security context graph containing relationships among assets, services, controls, exposures, and trust boundaries.

The security context is then used for control and risk reasoning. Controls are characterized according to their observed state, and risk scenarios are generated from the relationships and observations available in the context. Separate verification stages evaluate whether a scenario or remediation state is supported by subsequent technical evidence.

The outputs of the two evidence paths are brought into an evidence-fusion stage. Fusion provides a common representation through which structured security facts and retrieved documents are considered together while retaining provenance and contextual attributes.

Fused evidence is subjected to semantic comparison and consensus analysis. Pairwise NLI determines whether evidence items support, contradict, or remain neutral toward one another. Property and version context qualifies apparent disagreement. A consensus measure provides an additional signal concerning the degree to which the evidence set is mutually consistent.

Trust is calibrated from multiple dimensions of evidence quality. The eight-dimensional formulation involves signals associated with retrieval, agreement, coverage, reasoning, freshness, evidence quality, relevance, and feedback. The calibrated result is used as an input to the response policy rather than being treated as an unrestricted authorization to generate.

The safety policy gate follows evidence reasoning and trust calibration. This ordering allows explicit conditions to constrain the language-model stage. Depending on the evaluated state, the system may permit generation, qualify the output with a warning, invoke a fallback path, or abstain/refuse when the evidence is contradictory or insufficiently trustworthy.

When generation is permitted, a unified language-model gateway routes the request to an available inference provider or local inference mechanism. The generated response is streamed to the client together with provenance citations and safety metadata, allowing the user to distinguish evidence-grounded output from restricted or fallback behavior.

Persistent services support the processing flow by retaining documents, vector representations, security assessments, graph-related information, user and activity data, and operational state. Cache and background-job mechanisms support repeated operations.

The knowledge subsystem monitors for knowledge gaps and supports the promotion of recurring queries into deterministic FAQ rules, with automated rollback of degraded rules.

7. Description of the invention in detail

7.1 Contradiction-aware NLI

7.1.1 Semantic contradiction detection

The NLI engine performs pairwise semantic comparison of evidence items. For each pair of evidence items (A, B), the engine applies a cross-encoder neural model (via the reranking subsystem) to obtain a raw semantic similarity score, then applies a structured decision hierarchy to determine the relationship: SUPPORTS, CONTRADICTS, RELATED, or UNRELATED.

Contradiction detection goes beyond simple textual disagreement. The engine extracts structured metadata from each evidence item including CVE identifiers (via regex CVE-YYYY-NNNNNNN), CWE identifiers (via regex CWE-NNN), file paths, and software version strings (via regex v?N.N.N). It also extracts security properties using regex-based pattern matching across seven defined security property types: AUTHENTICATION, AUTHORIZATION, ENCRYPTION, INPUT_VALIDATION, SECRET_MANAGEMENT, CSRF_SESSION, and MEMORY_COMMAND. Each property is classified as either VULNERABLE or SAFE with an associated confidence and evidence span.

7.1.2 Distinguishing genuine from apparent contradiction

The central technical problem is that two statements can appear inconsistent while referring to different security properties, configurations, releases, or upgrade states. NOVA addresses this through a six-rule decision hierarchy:

Rule 1 (Distinct CVEs): If evidence items refer to distinct CVE identifiers, they are classified as RELATED or UNRELATED rather than contradictory, regardless of opposing language.

Rule 2 (Version Upgrade Path / Temporal Context): If evidence items reference different software versions, or contain temporal keywords such as "historical", "deprecated", "formerly", "legacy", "previously", "was vulnerable", "prior to", or "superseded", and at least one item describes a vulnerability, the pair is classified as RELATED with a contradiction_type of HISTORICAL or VERSION. This prevents a version upgrade path from being treated as an active conflict.

Rule 3 (Active Property Contradiction): If evidence items exhibit opposing vulnerability status assertions (one asserting VULNERABLE, the other asserting SAFE) on the same security property, component, or version scope, and the disagreement is not explained by remediation guidance, the pair is classified as CONTRADICTS with a contradiction_type of ACTIVE. This represents a genuine evidence conflict requiring safety intervention.

Rule 4 (Remediation Guidance): If one evidence item provides remediation guidance (identified by phrases such as "to fix", "upgrade to", "how to remediate"), the pair is classified as SUPPORTS rather than contradictory, because remediation advice is complementary to vulnerability assertions rather than contradictory.

Rule 5 (Domain Overlap): If evidence items share domain context (detected through domain keyword overlap) without explicit entailment or contradiction, they are classified as RELATED.

Rule 6 (Default): Otherwise, evidence items are classified as UNRELATED.

7.1.3 Property-aware and version-aware disambiguation

Property-aware disambiguation operates by extracting security properties from each evidence text and comparing their types and states. For instance, if evidence A asserts "authentication is bypassed" (property: AUTHENTICATION, state: VULNERABLE) and evidence B asserts "encryption uses TLS 1.3" (property: ENCRYPTION, state: SAFE), these concern different security properties and are therefore not contradictory, even though one discusses a vulnerability and the other discusses a security control.

Version-aware disambiguation operates by extracting software version strings (e.g., "v1.0", "v1.2", "3.8.1") using regular-expression matching. If both evidence items contain version identifiers but those versions differ, the engine classifies the relationship as a version upgrade path rather than a contradiction. If both items contain the same version, version disambiguation does not apply and the engine proceeds to evaluate whether the opposing assertions constitute an active contradiction.

7.1.4 Implicit contradiction handling

Implicit contradictions arise when evidence items do not contain explicit opposing keywords but carry metadata that conflicts. The engine handles this through scope matching: if two items share the same CVE, CWE, file path, or security property type but describe opposing states, the contradiction is surfaced even if the natural language alone would not trigger a generic NLI contradiction label. This is particularly important for structured security evidence where the conflicting information may be encoded in metadata fields rather than prose.

7.1.5 Technical examples

Example 1 - Version disambiguation: Evidence A states "OpenSSL 1.0.2 is vulnerable to Heartbleed (CVE-2014-0160)." Evidence B states "OpenSSL 1.1.1 has patched CVE-2014-0160." A generic NLI model would classify these as contradictory because one says "vulnerable" and the other says "patched." NOVA's engine extracts versions 1.0.2 and 1.1.1, detects they are different, and classifies the pair as RELATED with contradiction_type=VERSION. The system correctly interprets this as a version upgrade path.

Example 2 - Active contradiction: Evidence A states "The /api/users endpoint requires admin authentication." Evidence B states "The /api/users endpoint allows unauthenticated access." Both reference the same endpoint and the same security property (AUTHENTICATION), but with opposing states (SAFE vs. VULNERABLE). The engine classifies this as CONTRADICTS with contradiction_type=ACTIVE. This signals a genuine evidence conflict requiring downstream safety intervention.

Example 3 - Historical context: Evidence A states "The application was previously vulnerable to SQL injection." Evidence B states "The application now sanitizes all user input." The temporal keyword "previously" causes Rule 2 to classify this as RELATED with contradiction_type=HISTORICAL rather than an active contradiction.

Example 4 - Different property types: Evidence A states "Authentication is bypassed on the admin panel." Evidence B states "All database queries use parameterized statements." These concern different security properties (AUTHENTICATION vs. INPUT_VALIDATION) and do not contradict each other. The engine correctly classifies them as RELATED or SUPPORTS rather than CONTRADICTS.

7.2 Multi-dimensional evidence trust calibration

7.2.1 The eight evidence-quality dimensions

The trust calibrator computes an eight-dimensional confidence vector C:

C = [C_retrieval, C_agreement, C_citation, C_reasoning, C_freshness, C_hallucination_risk, C_reliability, C_feedback]

(a) C_retrieval: Semantic retrieval similarity score from the vector store, measuring how closely the retrieved evidence matches the query.

(b) C_agreement: Pairwise consensus agreement score computed by the NLI consensus engine across the NxN evidence relationship matrix. Measures how consistently the retrieved evidence items support one another.

(c) C_citation: Citation coverage score, measuring how well the evidence basis covers the information needed for the response.

(d) C_reasoning: Structural evidence completeness and reasoning support score, computed from the number of candidate evidence items relative to the requested top-k, with adjustments for analysis complexity (standard vs. deep_analysis).

(e) C_freshness: Exponential time-decay freshness score: C_freshness = exp(-lambda * age_days), where lambda = 0.005 and age_days is the number of days since the evidence was created or last updated. Recent evidence receives higher freshness scores; evidence from one year ago produces C_freshness approximately equal to 0.16.

(f) C_hallucination_risk: Evidence support mismatch risk, computed as: C_hallucination_risk = 1.0 - (0.5 C_retrieval + 0.3 C_agreement + 0.2 * C_citation). Higher values indicate greater risk of unsupported generation.

(g) C_reliability: Weighted average source reliability based on the evidence source type. The source reliability weights implemented in the system are: faq_axiom=1.0, security_finding=0.95, official_doc=0.95, knowledge_doc=0.85, user_doc=0.80, web_search=0.70.

(h) C_feedback: User feedback score reflecting historical user satisfaction with similar evidence or responses.

7.2.2 Platt scaling calibration

The eight dimensions are combined through a weighted logistic function (Platt scaling) into a single calibrated trust probability:

logit = 2.5C_retrieval + 2.0C_agreement + 1.5C_citation + 1.0C_reasoning + 1.0C_freshness - 3.0C_hallucination_risk + 1.0C_reliability + 0.5C_feedback - 2.8

P(Trust) = 1.0 / (1.0 + exp(-logit))

This produces a calibrated trust probability P(Trust) in [0, 1] that reflects the combined quality of the evidence basis. The negative coefficient on C_hallucination_risk means that high hallucination risk substantially reduces the trust score. The bias term -2.8 requires positive evidence across multiple dimensions for the trust score to exceed 0.50.

7.2.3 Why a single score is insufficient

A single cosine similarity or model-confidence score cannot capture the multi-faceted nature of evidence quality. A document may have high semantic similarity to the query but may be stale (low freshness), may contradict other evidence (low agreement), or may originate from an unreliable source (low reliability). The eight-dimensional formulation allows the system to detect situations where one dimension is strong but others are weak, preventing a misleadingly high aggregate confidence.

7.2.4 Distinction between trust estimation and safety decision

The calibrated trust score P(Trust) is an input to the safety decision, not the decision itself. Trust calibration estimates the probability that the evidence basis supports a correct answer. The safety gate (Section 7.3) makes the final authorization decision. This separation ensures that statistical trust cannot by itself authorize generation when evidence contradictions exist.

> Example: High similarity, stale evidence. A query about "current TLS configuration" retrieves a document with high cosine similarity (C_retrieval=0.92) but from 2019 (C_freshness=0.12). Despite high retrieval similarity, the low freshness dimension reduces the overall trust score, causing the system to issue a qualified warning rather than an unqualified answer.
>
> Example: Multiple sources agreeing. Five independent evidence items from different sources (security finding, knowledge document, official documentation) all assert the same conclusion. C_agreement=0.95, C_reliability=0.92, C_citation=0.90. The high agreement across multiple reliable sources produces a high trust score, supporting confident generation.

7.3 Hard / Explainable safety gate

7.3.1 Separation from trust calibration

The safety gate is architecturally separate from the trust calibrator. This separation is the central safety mechanism of the system. Trust calibration produces a statistical probability; the safety gate applies discrete policy conditions. The two operate sequentially: trust is computed first, then the safety gate evaluates whether the evidence conditions permit generation.

For security-related queries, the effective trust threshold is raised to a minimum of 0.75 (compared to the standard 0.70), providing additional conservatism for security-sensitive decisions.

7.3.2 Four-outcome decision logic

The safety gate implements the following decision logic:

> (a) GENERATE: The trust score exceeds the effective threshold AND no critical evidence contradictions exist. The system generates a response with provenance citations.
>
> (b) GENERATE_WITH_WARNING: The trust score exceeds the effective threshold but one or more warning conditions are present (e.g., elevated hallucination risk, aged evidence, low retrieval similarity). The system generates a response with explicit safety warnings attached.
>
> (c) FALLBACK_WEB: The trust score is below the threshold, or a critical evidence contradiction is detected (C_agreement <= 0.20), and web fallback is enabled. The system redirects to an external search mechanism rather than generating from potentially contradictory evidence.
>
> (d) ABSTAIN / REFUSE: The trust score is below the threshold or a critical contradiction is detected, and web fallback is not available. The system explicitly refuses to generate and explains why.

7.3.3 Why high confidence cannot bypass the gate

The critical technical mechanism is that the hard contradiction override is structurally decoupled from the trust probability. Even if P(Trust) = 0.97, if C_agreement <= 0.20 and the contradiction type is ACTIVE, the gate forces FALLBACK_WEB or ABSTAIN. This prevents the following failure mode present in conventional systems: a high-confidence retrieval result that matches the query well but conflicts with other retrieved evidence is nevertheless used for generation because the confidence threshold is exceeded.

However, the gate also considers the contradiction type through the NLI disambiguation. If the low agreement score results from a HISTORICAL or VERSION contradiction rather than an ACTIVE contradiction, the gate permits generation because the disagreement reflects temporal evolution or a version upgrade path rather than genuine conflicting information.

> Example - Safety gate blocking: The system retrieves five evidence items about a server's authentication configuration. Three items assert "authentication is required" and two items assert "the endpoint allows unauthenticated access." The NLI engine classifies the opposing assertions as ACTIVE contradictions. C_agreement = 0.15. Even though P(Trust) = 0.95 (because retrieval similarity is high and sources are reliable),

the hard contradiction override forces FALLBACK_WEB. The system will not generate a potentially dangerous security recommendation from contradictory evidence.

Example - Safety gate permitting through version context: The system retrieves evidence about OpenSSL. One item states "OpenSSL 1.0.2 is vulnerable" and another states "OpenSSL 1.1.1 is patched." The NLI engine classifies this as contradiction_type=VERSION. Despite low surface agreement, the gate permits GENERATE because the disagreement reflects a version upgrade path, not a genuine evidence conflict.

7.3.4 Provenance and explainability

The safety gate produces an auditable decision record (AuditableDecisionRecord) that captures:

- The input query
- Evidence IDs used in the decision
- The pairwise NLI relationship matrix
- Consensus metrics (agreement score, contradiction count)
- The full 8D confidence vector
- The Platt logit and trust score
- The policy trigger that determined the decision (CRITICAL_CONTRADICTION, SECURITY_QUERY_LOW_CONFIDENCE, LOW_RETRIEVAL_SIMILARITY, CONFIDENCE_THRESHOLD_OVERRIDE, or NORMAL_CONFIRMED)
- The final decision (GENERATE / GENERATE_WITH_WARNING / FALLBACK_WEB / ABSTAIN)
- An explanation payload with structured safety metadata

This record allows the reason for any generation, warning, fallback, or refusal decision to be inspected, audited, and explained. The explanation includes identification of the specific contradicting evidence pairs and the safety policy that was triggered.

7.4 Security context graph

7.4.1 Entities and relationships

The security context graph represents the following entity types and their relationships:

- Assets: Discovered application components, APIs, services, databases, and deployment boundaries, identified through asset discovery scanning.
- Observations: Structured facts about security-relevant conditions obtained from code analysis (route definitions, parameter handling, secrets, database access patterns), produced by AST parsing of the source repository.
- Evidence: Supporting technical material including source-code excerpts, configuration fragments, and static-analysis artifacts.
- Controls: Security mechanisms detected through AST inspection, including authentication middleware (RequireRole decorators), authorization checks (Depends() injections), input validation (Pydantic models), and query parameterization.
- Risks: Identified risk conditions derived from control absence, insecure patterns, or policy violations.
- Trust Boundaries: Logical boundaries between components with different trust levels (e.g., public API surface vs. internal services).

7.4.2 Why isolated findings are insufficient

A conventional vulnerability scanner produces isolated findings: "SQL injection in file X at line Y." Without contextual relationships, the system cannot determine: (a) whether the affected endpoint is protected by authentication middleware; (b) whether input validation exists elsewhere in the call chain; (c) whether the finding was previously remediated; (d) whether the risk affects a trust boundary between public and internal services. The security context graph provides these contextual relationships, enabling situated security intelligence rather than disconnected alerts.

7.4.3 How graph context improves security reasoning

The intelligence orchestrator uses graph context to coordinate control analysis and risk reasoning. When evaluating a

potential vulnerability, the system can traverse the graph to identify related controls, assess whether those controls mitigate the risk, and determine the trust boundary context. This contextual reasoning transforms isolated findings into situated security intelligence.

7.4.4 Posture and trend analysis

The posture trend engine computes temporal security posture by comparing assessment snapshots across time. The posture delta is computed as: dS = S_t - S_{t-1}, where S_t is the security posture score at time t. Risk evolution is classified into four categories:

- NEW_RISK: A risk condition that was not present in the previous snapshot.
- PERSISTENT_RISK: A risk condition that remains unchanged between snapshots.
- RESOLVED_RISK: A risk condition that was present in the previous snapshot but is no longer present.
- RESURFACED_RISK: A risk condition that was previously resolved but has reappeared.

This classification enables the system to track whether the overall security condition is improving, degrading, or oscillating, providing temporal intelligence beyond single-point-in-time scanning.

7.5 Risk scenario generation and verification

7.5.1 Separation of generated scenarios from verified conditions

The system maintains an explicit separation between hypothesized risk scenarios and verified security conditions. The security intelligence pipeline implements a five-stage explicit lifecycle with machine-verifiable state transitions:

Stage 1 - OBSERVED: The observation collector parses source code using Python AST analysis to extract structural facts about routes, parameters, inputs, secrets, and database queries. Each observation carries an asset_id, version_hash, timestamp, provenance_chain, and transition_condition.

Stage 2 - CONTROL_EVALUATED: The control analyzer inspects AST nodes for security mechanisms (RequireRole decorators, Depends() injections, Pydantic model validations). Each evaluation carries parent_observation_ids linking it to the originating observations.

Stage 3 - RISK_CANDIDATE: The risk scenario engine synthesizes observations and control evaluations into candidate risk hypotheses. These are explicitly marked as candidates, not verified conditions. Each inference carries parent_control_ids preserving the provenance chain.

Stage 4 - VERIFIED: The scenario verifier gate cross-references candidate hypotheses against control evaluations. Only candidates supported by verification evidence transition to the VERIFIED state.

Stage 5 - ASSESSMENT: Verified assessments are persisted to the security assessment store and made available to the evidence fusion engine for downstream reasoning.

7.5.2 Contextual evidence for verification

Verification uses the control evaluations from Stage 2 and the security context graph relationships. A risk candidate is verified by examining whether the hypothesized condition is supported by the observed controls (or absence of controls) and the architectural context in which the observation occurs.

Example: The risk engine generates a candidate hypothesis: "The /api/admin/users endpoint may be vulnerable to unauthorized access." The scenario verifier checks the control evaluation for that endpoint and finds a RequireRole("admin") decorator confirmed by AST analysis. Because the control is present and confirmed, the risk candidate is not verified as a supported risk -- the control mitigates the hypothesized vulnerability.

Example: The risk engine generates a candidate: "The /api/public/upload endpoint accepts unsanitized file paths." The scenario verifier checks for input validation controls and finds no Pydantic model or sanitization function on the endpoint. The candidate transitions to VERIFIED because the absence of controls supports the risk hypothesis.

7.6 Remediation verification

7.6.1 Recommendation is not proof

A critical distinction in the system is that generating a remediation recommendation does not demonstrate that the

underlying technical condition has been corrected. The remediation verifier implements three tiers of verification using AST-oriented code analysis:

Tier 1 - Textual Presence: Checks whether remediation-related keywords (RequireRole, Depends(, Pydantic, sanitiz, select()) appear in the updated code snippet. This is a necessary but insufficient condition for verified remediation.

Tier 2 - Structural Implementation: Parses the updated code using Python AST analysis (ast.parse()) to verify that security controls are structurally present in the code. Specifically, the verifier walks the AST tree looking for FunctionDef or AsyncFunctionDef nodes with decorator lists containing Depends() or RequireRole patterns. This goes beyond keyword matching to confirm that the control is syntactically correct and structurally placed on the correct function.

Tier 3 - Control Effectiveness: Evaluates whether the structurally present controls are configured to be effective. For example, a RequireRole decorator specifying "admin" or "user" is considered effective, whereas an empty or misconfigured decorator is not. This tier confirms that the control not only exists structurally but is configured to provide the intended security protection.

7.6.2 Before-remediation and after-remediation state

The remediation verifier operates on the code state after remediation. It receives an assessment_id identifying the security condition to verify and a code_snippet representing the current code state. The verifier returns a structured result including:

- Status: VERIFIED_FIXED (all tiers passed) or OPEN (verification failed)

- Verification tiers: textual_presence (boolean), structural_implementation (boolean), control_effectiveness (boolean)

- Controls detected: authorization_middleware (boolean), input_validation (boolean), query_parameterization (boolean)

- Verification summary: a human-readable explanation of the verification outcome

Example: An assessment identifies "Missing authentication on /api/admin." The developer adds @Depends(RequireRole("admin")) to the endpoint function. The remediation verifier: (1) finds "RequireRole" textually present (Tier 1 pass); (2) parses the AST and confirms a FunctionDef decorator containing "RequireRole" (Tier 2 pass); (3) confirms the decorator argument contains "admin" (Tier 3 pass). Status transitions to VERIFIED_FIXED.

Example: An assessment identifies "SQL injection in search function." The developer adds a comment "# TODO: fix SQL injection" but does not change the query code. The remediation verifier: (1) does not find parameterization keywords like select() or bind (Tier 1 fail); (2) AST parsing finds no structural security changes (Tier 2 fail). Status remains OPEN. The system correctly distinguishes an intended fix from an actual fix.

The verified remediation state feeds back into the posture trend engine and the security context graph, so that a verified fix is reflected in the temporal posture analysis as a RESOLVED_RISK classification.

7.7 Dual-track evidence acquisition and evidence fusion

7.7.1 Two evidence paths

NOVA acquires evidence through two complementary paths:

Track A - Unstructured Enterprise Knowledge (RAG): Documents (PDF, DOCX, MD, TXT) are ingested, processed through heading-aware semantic chunking that preserves section structure and context, embedded using FastEmbed (all-MiniLM-L6-v2 384-dimensional embeddings), and stored in a pgvector HNSW dense vector index. Retrieved knowledge chunks provide policy, procedural, and contextual information.

Track B - Structured Security Intelligence: The five-stage security intelligence pipeline (described in Section 7.5) produces verified security assessments based on AST code analysis. These assessments carry structured metadata including asset_id, version_hash, provenance_chain, and verification_state.

7.7.2 Evidence fusion with provenance preservation

The evidence fusion engine normalizes heterogeneous sources into a unified evidence representation (UnifiedEvidenceItem). Each item preserves:

- source_type (knowledge_doc, security_finding, faq_axiom, web_search)
- source_id (unique evidence identifier)
- similarity_score and rerank_score (retrieval quality metrics)
- reliability_weight (source-type-based weight from the calibrator's weight table)
- provenance (origin metadata dictionary)
- asset_id and version (scope and version context for the evidence)
- verification_state (VERIFIED, OBSERVED, UNVERIFIED)
- parent_evidence_ids and derived_evidence_ids (evidence lineage tracking)
- temporal_validity (ACTIVE, HISTORICAL, DEPRECATED, STALE)
- security_property (the security property type if applicable)

Cross-scanner confidence boosting is applied when the same condition is detected by multiple independent sources: C_finding = 1.0 - PROD(1.0 - c_i), where c_i is the base confidence of each independent detection. For example, if two independent scanners each detect a finding with 0.85 confidence, the fused confidence is 1.0 - (0.15 * 0.15) = 0.9775.

7.7.3 Distinction from standard RAG

Standard RAG retrieves text chunks based on similarity, ranks them, and passes them to a language model for generation. NOVA's dual-track approach differs in three significant respects:

(1) It acquires structured technical evidence from code analysis alongside unstructured knowledge from documents, rather than relying exclusively on document retrieval.

(2) It preserves provenance, verification state, evidence lineage (parent_evidence_ids, derived_evidence_ids), temporal validity, and security property through the fusion stage, rather than treating all retrieved items as undifferentiated text chunks.

(3) The fused evidence undergoes pairwise NLI contradiction analysis and eight-dimensional trust calibration before any generation occurs, rather than being passed directly to a language model.

Example: A security query about authentication on the /api/admin endpoint retrieves both: Track A evidence (a company security policy document stating "all admin endpoints must require multi-factor authentication") and Track B evidence (an AST observation showing a RequireRole("admin") decorator on the endpoint but no MFA middleware). The fusion engine combines both items with their respective source types and provenance. The NLI engine identifies a partial gap between the policy requirement (MFA) and the observed control (role-based auth without MFA), enabling the system to produce a nuanced response acknowledging both the existing control and the policy gap.

7.8 Knowledge-gap detection and knowledge evolution

7.8.1 Knowledge gaps as structured signals

When the existing knowledge or retrieval path is insufficient to produce a satisfactory evidence-grounded result, the system records a knowledge gap. Low-confidence queries, unanswered interactions, and fallback responses generate structured gap records that capture the query, the retrieval results (or lack thereof), and the trust evaluation that triggered the gap. These gaps are stored in the search analytics subsystem for subsequent analysis.

7.8.2 Gap clustering and draft FAQ synthesis

The knowledge gap analytics system clusters frequently occurring gaps using frequency-based analysis. When a cluster of related gaps reaches sufficient density, the system generates a draft FAQ rule (is_draft=True) as a candidate for human review. Draft rules are not active until explicitly promoted through human curation. This prevents unverified automated content from entering the deterministic response path.

7.8.3 Deterministic Stage-0 handling

When a query matches an active (promoted and human-curated) FAQ rule, the system handles it through a

deterministic fast path (Stage 0) that produces a response without invoking the full retrieval and generation pipeline. This converts recurring well-understood queries from expensive operations into near-instantaneous deterministic responses.

7.8.4 Closed-loop feedback and rollback

The system monitors the effectiveness of active FAQ rules. If a rule generates excessive fallback responses (fallback count >= 5), the closed-loop feedback monitor automatically deactivates the rule to prevent degraded behavior. This self-healing mechanism ensures that promoted rules that become inaccurate or stale are automatically rolled back rather than continuing to produce incorrect responses.

7.8.5 Distinction from earlier AEKOF concepts

The current NOVA implementation focuses on the FAQ-based deterministic fast path, gap clustering, and closed-loop rollback mechanisms present in the current repository. The earlier AEKOF design contained additional conceptual mechanisms for more extensive self-healing retrieval evolution. This disclosure is limited to the implemented mechanisms as verified in the repository.

7.9 Implementation evidence

Representative implementation locations in the current repository include:

- backend/app/services/security_intelligence/ (asset_discovery_service.py, observation_collector.py, control_analyzer.py, risk_scenario_engine.py, scenario_verifier.py, remediation_verifier.py, posture_trend_engine.py, evidence_provider.py, context_graph_builder.py, intelligence_orchestrator.py, security_explanation_engine.py)
- backend/app/api/v1/security_intelligence.py
- backend/app/models/security_intelligence.py
- backend/app/services/ai/nli_engine.py
- backend/app/services/ai/consensus_engine.py
- backend/app/services/assistant/evidence_fusion.py
- backend/app/services/search_analytics/calibrator.py
- backend/app/services/knowledge_base/graph_builder.py
- backend/app/services/faq_service.py

The security-intelligence directory contains the dedicated asset discovery, observation, evidence, control, explanation, orchestration, scenario, remediation, posture, and context-graph mechanisms. The AI and assistant services provide the NLI, consensus, trust-calibration, and evidence-fusion mechanisms, while the knowledge-base services provide graph and retrieval support.

8. Experimental validation results

8.1 Validation infrastructure

The repository contains dedicated validation infrastructure covering the principal mechanisms of the invention. The identified validation areas include:

- Security-intelligence service testing
- NLI consensus and contradiction testing, including implicit and version/property contradiction cases
- Evidence-fusion integration testing
- Safety-gate explainability testing
- Temporal posture-engine testing
- Canonical demo-workflow testing
- NOVA core-feature testing
- Scientific or forensic validation scripts

The repository reconnaissance identified approximately 25 pytest suites and 9 validation or forensic evaluation scripts. These demonstrate that validation infrastructure exists for the principal mechanisms.

8.2 Patent-differentiating interaction tests

A dedicated 14-scenario interaction test suite (test_patent_differentiating_interactions.py) verifies the core patent-differentiating mechanisms:

Tests 1-4: NLI property/version disambiguation (active contradiction detection, version upgrade path classification, historical context classification, different-property non-contradiction).

Tests 5-6: Three-tier remediation verification (successful remediation with AST verification, failed remediation detection).

Test 7: Temporal posture and risk evolution classification (NEW_RISK, PERSISTENT_RISK, RESOLVED_RISK, RESURFACED_RISK).

Test 8: Dual-track evidence fusion with provenance preservation.

Test 9: Pairwise consensus matrix computation.

Test 10: 8D trust calibration with Platt scaling.

Test 11: Hard safety gate contradiction override (C_agreement <= 0.20 forces FALLBACK_WEB despite high trust score).

Test 12: Auditable decision record generation.

Tests 13-14: End-to-end security intelligence pipeline state transitions (OBSERVED -> CONTROL_EVALUATED -> RISK_CANDIDATE -> VERIFIED -> ASSESSMENT).

8.3 Test results

The full pytest suite produces 348 passed tests with 0 failures. The backend application imports successfully. The frontend build completes with 0 errors.

8.4 Quantitative performance results

Numerical accuracy percentages, comparative performance improvements, latency measurements, and production-scale benchmark results are not claimed in this disclosure because they have not been produced from executed experimental runs. For final submission:

To be populated with executed experimental results including: retrieval precision/recall, NLI contradiction classification accuracy and false-positive/false-negative rates, trust calibration reliability (calibration curves), safety gate false-positive and false-negative rates, end-to-end response latency, posture trend analysis accuracy, and remediation verification accuracy.

9. What aspect(s) of the invention need(s) protection?

The following aspects are identified as candidates for intellectual-property protection, with the strongest candidates listed first:

(a) A contradiction-analysis workflow combining pairwise NLI with property-aware and version-aware disambiguation to distinguish genuine evidence conflict from contextual differences. The specific six-rule decision hierarchy extracting seven security property types with two states, software version comparison, and temporal keyword analysis to produce explicit contradiction type classifications (ACTIVE, HISTORICAL, VERSION, RESOLVED, STALE, UNVERIFIED).

(b) A multi-dimensional evidence-trust calibration mechanism using an eight-dimensional confidence vector combined through Platt scaling, coupled to a structurally decoupled hard safety policy gate. The gate enforces output refusal or fallback when critical evidence contradictions are detected (C_agreement <= 0.20 with ACTIVE contradiction type), regardless of the statistical trust probability, while permitting generation when low agreement results from historical or version-related context.

(c) A context-aware security-intelligence workflow combining asset and observation evidence, a security context graph, risk-scenario generation and verification with explicit lifecycle state transitions (OBSERVED -> CONTROL_EVALUATED -> RISK_CANDIDATE -> VERIFIED -> ASSESSMENT -> TEMPORAL_POSTURE), and three-tier remediation verification (textual presence, structural AST implementation, control effectiveness).

(d) A mechanism for fusing structured security or static-analysis evidence with retrieved unstructured knowledge

for evidence-grounded security reasoning, preserving provenance, verification state, evidence lineage (parent and derived evidence IDs), temporal validity, and security property through a unified evidence representation.

(e) A feedback mechanism in which detected knowledge gaps inform knowledge and retrieval evolution, including deterministic Stage-0 handling of recognized queries and closed-loop rollback of degraded FAQ rules when fallback count exceeds a threshold.

(f) An auditable decision record mechanism that captures the complete causal chain from evidence acquisition through NLI analysis, consensus computation, eight-dimensional trust calibration, and safety gate decision, producing a machine-readable and human-inspectable explanation of why a particular response was generated, qualified, redirected, or refused.

For patent drafting, the protection analysis should concentrate on the concrete system architecture, processing sequence, data structures, decision gates, and interactions between components. Generic claims to AI, RAG, NLI, knowledge graphs, or security scanning alone should not be assumed to be novel.

The earlier technical assessment particularly identifies the NLI/property-version engine, decoupled multi-dimensional trust calibration with hard safety gating, and context-aware security intelligence with remediation verification as leading candidates for focused novelty analysis.

The potentially important contribution is the specific interaction, ordering, and data-dependency chain among these mechanisms, not the individual techniques in isolation.

10. Technology readiness level

Research | Development | Deployment

TRL 1 | TRL 2 | TRL 3 | TRL 4 | TRL 5 | TRL 6 | TRL 7 | TRL 8 | TRL 9

[X] TRL 4

TRL 4 - Technology validated in a lab.

Justification: The core mechanisms are implemented in a functional repository with 348 passing tests covering all principal invention aspects. The system has been validated through unit tests, integration tests, and dedicated patent-differentiating interaction tests in a development/lab environment. Production-scale deployment, performance benchmarking under load, and operational environment validation have not yet been conducted.

----------*END OF THE DOCUMENT*----------